Project - High Level Design

On

Deploy Manufacturing App to Kubernetes

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: AAI-12

Industry Mentor Name: Prof. Akshay Saxena

University Mentor Name:

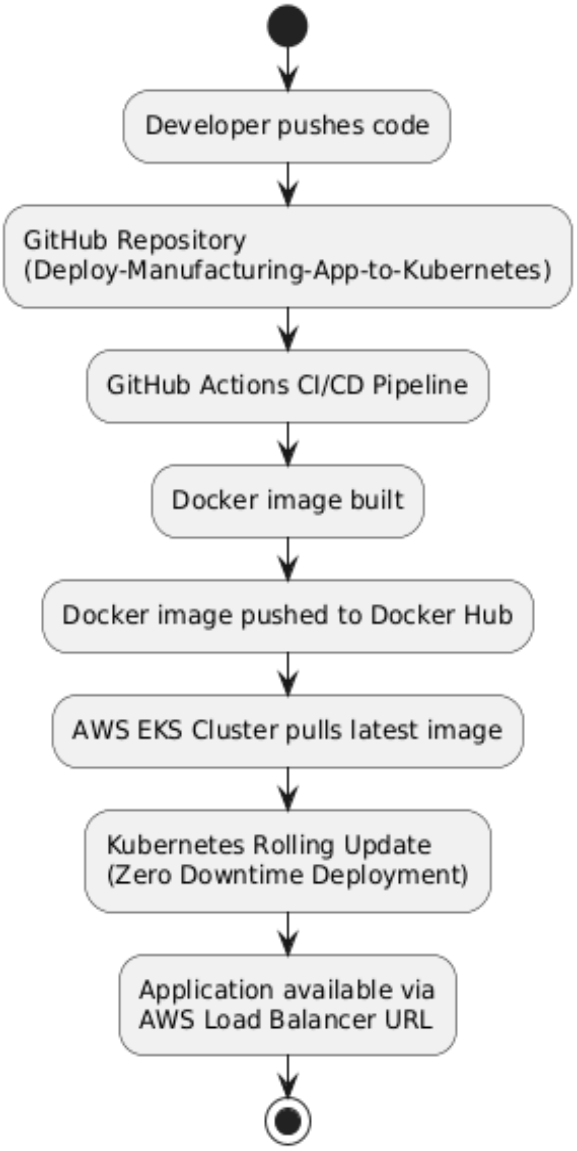
Academic Year: 2025-2026

1. Objective

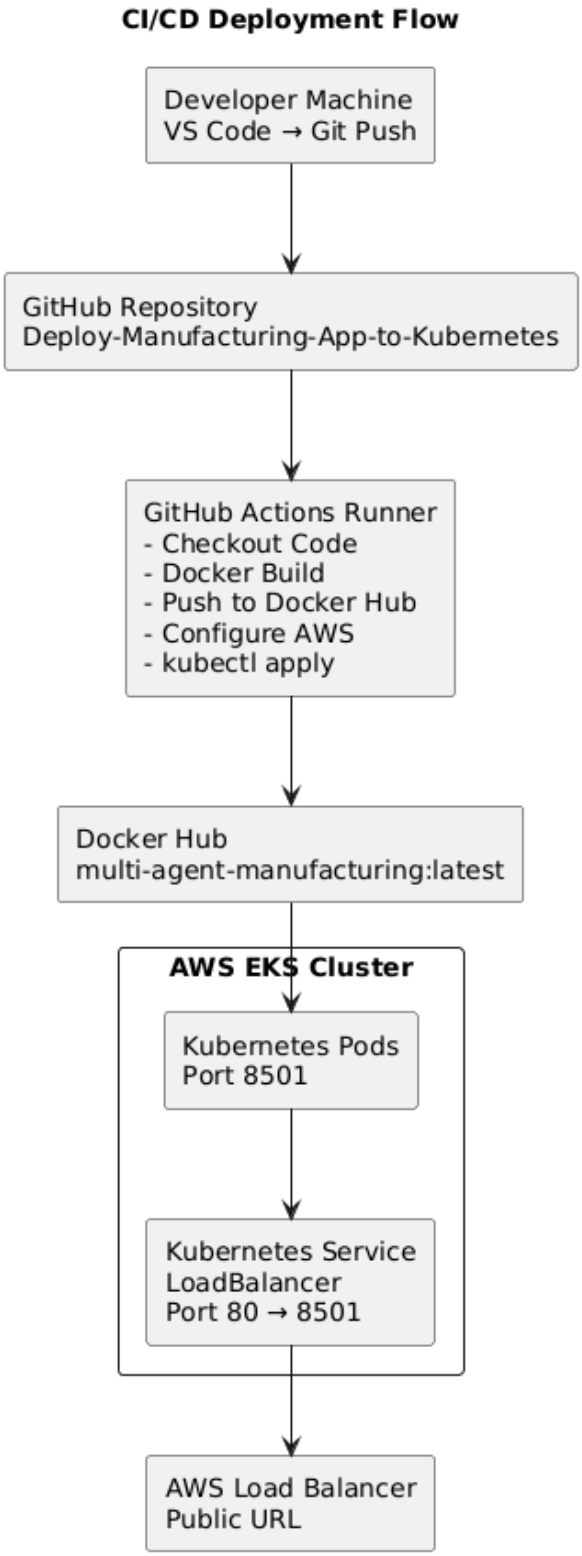
To containerize and deploy the Multi-Agent Manufacturing Streamlit application onto a cloud-based Kubernetes cluster (AWS EKS) using Docker, with a fully automated CI/CD pipeline via GitHub Actions. The deployment implements zero-downtime Rolling Updates to ensure seamless version transitions without service interruption.

2. System Architecture

2.1 High Level Flow

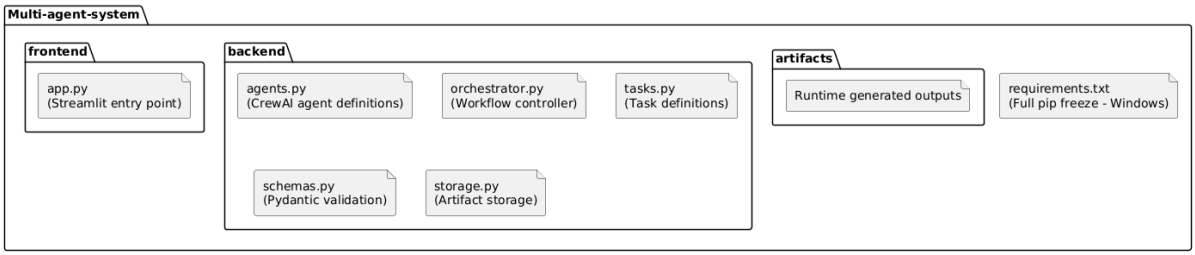


2.2 Component Interaction

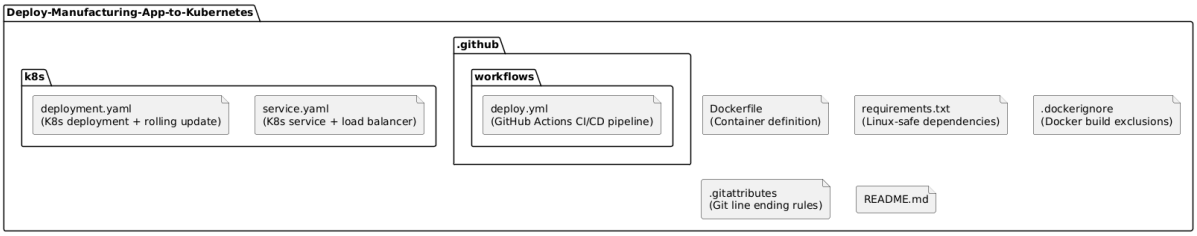


3. Repository Structure

3.1 Source Repository



3.2 DevOps Repository



4. Component Level Design

4.1 Dockerfile

Base Image: python:3.11-slim

Build Strategy: Clone source at build time

Step	Instruction	Purpose
1	FROM python:3.11-slim	Lightweight Linux base image
2	apt-get install build-essential curl git	System dependencies
3	COPY requirements.txt /requirements.txt	Stage deps before clone
4	git clone Multi-agent-system .	Pull AI app source code
5	pip install -r requirements.txt	Install Python packages
6	mkdir -p /app/artifacts	Runtime output directory

Key Design Decision: requirements.txt is copied to /requirements.txt (not /app) before the git clone runs into /app. This avoids Git refusing to clone into a non-empty directory.

4.2 deployment.yaml

Kind: Deployment **API Version:** apps/v1

Parameter	Value	Reason
replicas	2	High availability, always 2 pods running
strategy.type	RollingUpdate	Zero downtime updates
maxSurge	1	Spin up 1 extra pod before killing old one
maxUnavailable	0	Never reduce below 2 healthy pods
imagePullPolicy	Always	Ensures latest image on every deploy
containerPort	8501	Streamlit default port
memory request	512Mi	Minimum guaranteed memory per pod
memory limit	1Gi	Maximum memory cap per pod
cpu request	250m	0.25 CPU core guaranteed
cpu limit	500m	0.5 CPU core maximum

Readiness Probe:

Parameter	Value
Path	/_stcore/health
initialDelaySeconds	30
periodSeconds	10
failureThreshold	3

Liveness Probe:

Parameter	Value
Path	/_stcore/health
initialDelaySeconds	60
periodSeconds	15
failureThreshold	3

4.3 service.yaml

Kind: Service **Type:** LoadBalancer

Parameter	Value	Reason
type	LoadBalancer	AWS auto-provisions ELB for external access
port	80	Standard HTTP port for users
targetPort	8501	Forwards to Streamlit inside pod
selector	app: multi-agent-manufacturing	Routes to correct pods

4.4 GitHub Actions Pipeline (deploy.yml)

Trigger: Push to main branch

Step	Action	Details
1	Checkout code	actions/checkout@v3
2	Login to Docker Hub	Uses DOCKER_USERNAME + DOCKER_PASSWORD secrets
3	Build Docker image	docker build -t tiwariom/multi-agent-manufacturing:latest
4	Push to Docker Hub	docker push tiwariom/multi-agent-manufacturing:latest
5	Configure AWS credentials	Uses AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY secrets
6	Update kubeconfig	aws eks update-kubeconfig --name multi-agent-cluster
7	Deploy to EKS	kubectl apply -f k8s/deployment.yaml
8	Verify rollout	kubectl rollout status deployment/multi-agent-manufacturing

GitHub Secrets Required:

Secret	Value
DOCKER_USERNAME	tiwariom
DOCKER_PASSWORD	Docker Hub access token
AWS_ACCESS_KEY_ID	From AWS IAM
AWS_SECRET_ACCESS_KEY	From AWS IAM
AWS_REGION	ap-south-1

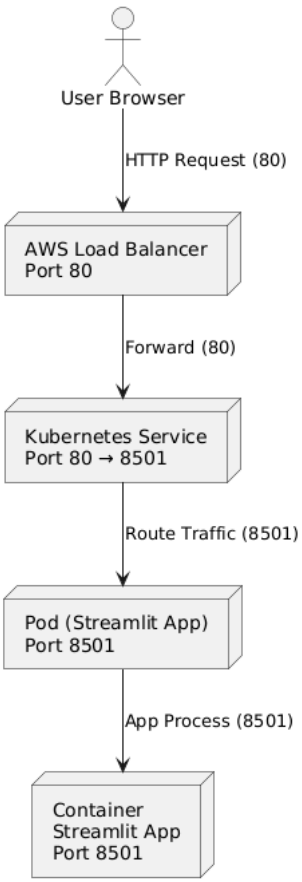
5. AWS Infrastructure Design

Component	Service	Configuration
Kubernetes Cluster	AWS EKS	Version 1.29, region ap-south-1
Worker Nodes	EC2 t3.small	Min: 1, Max: 2, Desired: 2
Container Registry	Docker Hub	Public repository
Load Balancer	AWS ELB	Auto-provisioned by K8s Service
CI/CD	GitHub Actions	Triggered on push to main

6. Environment Variables

Variable	Set In	Purpose
STREAMLIT_SERVER_PORT	Dockerfile	Port Streamlit listens on
STREAMLIT_SERVER_ADDRESS	Dockerfile	Bind to all interfaces
STREAMLIT_SERVER_HEADLESS	Dockerfile	Disable browser auto-open
CREWAI_DISABLE_TELEMETRY	Dockerfile	Disable CrewAI tracking
OTEL_SDK_DISABLED	Dockerfile	Disable OpenTelemetry
GOOGLE_API_KEY	Runtime (UI)	Entered by user in Streamlit sidebar

7. Port Mapping



8. Security Considerations

Concern	Approach
API Keys	Never stored in image entered via Streamlit UI at runtime
Docker Hub credentials	Stored as GitHub Secrets, never in code
AWS credentials	Stored as GitHub Secrets, IAM user with minimum permissions
Image scanning	Docker Hub auto-scan on push